OS LAB 10 SUBMISSION

Name: Yash Raj
Roll Number: 24k-0737

Task01:
Code:

```c
#include<unistd.h>
#include<stdio.h>
#include<stdlib.h>
#include<pthread.h>
#include<semaphore.h>

sem_t wrt;
sem_t queue;
pthread_mutex_t mutex;
int readcount =0;
int data=0;

void*reader(void * arg){
int id = *(int*)arg;
sem_wait(&queue);
pthread_mutex_lock(&mutex);
readcount ++;
if(readcount == 1){
sem_wait(&wrt);
}
pthread_mutex_unlock(&mutex);
sem_post(&queue);
printf("Reader having id = %d is reading data = %d",id,data);
sleep(1);

pthread_mutex_lock(&mutex);
readcount--;
if(readcount == 0){
sem_post(&wrt);
}
pthread_mutex_unlock(&mutex);
return NULL;
}

void * writer(void *arg){
int id = *(int*)arg;
sem_wait(&queue);
sem_wait(&wrt);
data++;
 printf("Writer %d is writing data = %d\n", id, data);
 sleep(1);
 sem_post(&wrt);
 sem_post(&queue);
return NULL;
}
```

```
47
48 int main(){
49
50 pthread_t r[2], w[2];
51 int rid[2] = {1,2};
52 int wid[2] = {1,2};
53
54 pthread_mutex_init(&mutex,NULL);
55 sem_init(&wrt,0,1);
56 sem_init(&queue,0,1);
57 for (int i = 0; i < 2; i++) {
58         pthread_create(&r[i], NULL, reader, &rid[i]);
59     }
60
61 for (int i = 0; i < 2; i++) {
62         pthread_create(&w[i], NULL, writer, &wid[i]);
63     }
64     for (int i = 0; i < 2; i++) {
65         pthread_join(r[i], NULL);
66     }
67
68     for (int i = 0; i < 2; i++) {
69         pthread_join(w[i], NULL);
70     }
71
72     pthread_mutex_destroy(&mutex);
73     sem_destroy(&wrt);
74     sem_destroy(&queue);
75 }
```

Output:

```
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ gcc Task01.c -o H
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ ./H
Reader having id = 1 is reading data = 0Reader having id = 2 is reading data = 0Writer 1 is writing data = 1
Writer 2 is writing data = 2
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$
```

Task02:

```c
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<pthread.h>
#include<semaphore.h>

#define size 6
int buffer[size];
int in =0, out =0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void* producer(void * arg){
int id = *(int *)arg;
int item;


item = rand()%100;
sem_wait(&empty);
pthread_mutex_lock(&mutex);
buffer[in] = item;
printf("Producer %d produces %d\n",id,item);
in = (in + 1)% size;
pthread_mutex_unlock(&mutex);
sem_post(&full);
sleep(1);
return NULL;
}
void * consumer(void * arg){
int id = *(int *)arg;
int item;


sem_wait(&full);
pthread_mutex_lock(&mutex);
item = buffer[out];
  printf("Consumer %d consumed %d\n", id, item);
  out = (out + 1)%size;
  pthread_mutex_unlock(&mutex);
  sem_post(&empty);
sleep(2);

return NULL;
}
```

```c
int main(){


pthread_t p1 , p2, c1, c2;
int id1 = 1;
int id2 = 2;
int id3 = 1;
int id4 = 2;

sem_init(&empty,0,size);
sem_init(&full , 0,0);
pthread_mutex_init(&mutex,NULL);

    pthread_create(&p1, NULL, producer, &id1);
    pthread_create(&p2, NULL, producer, &id2);
    pthread_create(&c1, NULL, consumer, &id3);
    pthread_create(&c2, NULL, consumer, &id4);

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);
    pthread_join(c1, NULL);
    pthread_join(c2, NULL);

}
```

Output:

```
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ gcc Task02.c -o A
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ ./A
Producer 1 produces 83
Producer 2 produces 86
Consumer 1 consumed 83
Consumer 2 consumed 86
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$
```